

TALLER DE APPI

YURANI ALEXANDRA CASTILLO MARTINEZ

PROGRAMA: ANÁLISIS Y DESARROLLO DE SOFTWARE – SENA 3236085

ING. CARLOS JARAMILLO

22 E OCTUBRE 2025

INTRODUCCIÓN

El presente informe tiene como objetivo documentar el proceso de pruebas realizado sobre la API desarrollada para la gestión de usuarios. Para ello, se empleó la herramienta Postman, la cual permite enviar solicitudes HTTP y verificar el correcto funcionamiento de los endpoints implementados.

A lo largo del documento se presentan las evidencias de las pruebas realizadas, incluyendo solicitudes exitosas, manejo de errores y validación de respuestas del servidor. Además, se describen los pasos necesarios para ejecutar cada prueba, así como las configuraciones utilizadas dentro de Postman.

Este informe busca demostrar que los servicios creados cumplen con los requerimientos funcionales establecidos, garantizando su correcto comportamiento y facilitando la evaluación del trabajo solicitado.

1. Preparación del Ambiente de Desarrollo

Herramientas utilizadas:

- Node.js
- Visual Studio Code
- Npm
- Postman
- MongoDB Atlas / Compass

Dependencias instaladas:

- Express
- Mongoose
- Cors
- nodemon (dev)

2. Estructura del Proyecto Backend

```
BackEnd/  
├── index.js  
├── package.json  
├── database.js  
├── models/  
│   ├── usuario.model.js  
│   └── producto.model.js  
├── controllers/  
│   ├── usuario.controller.js  
│   └── producto.controller.js  
└── routes/  
    ├── usuario.routes.js  
    └── producto.routes.js
```

3. Base de Datos

Base de datos: empleados_085

Colecciones:

- Usuarios
- productos

4. API REST Implementadas

- **Usuarios:**
POST /api/usuarios
GET /api/usuarios
GET /api/usuarios/:id
PUT /api/usuarios/:id
DELETE /api/usuarios/:id
POST /api/usuarios/login
- **Productos:**
POST /api/productos
GET /api/productos
GET /api/productos/:id
PUT /api/productos/:id
DELETE /api/productos/:id

5. Pruebas con Postman (Capturas de Evidencia)

Evidencia del servidor backend ejecutándose correctamente.

En la consola se observa el mensaje 'server activo en el puerto 3000' y 'DB se encuentra conectada', lo cual confirma que la aplicación está enlazada con la base de datos MongoDB y lista para recibir peticiones a través de las rutas configuradas.

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

requireStack: [
  'C:\\Users\\marti\\OneDrive\\Escritorio\\SOFTWARE\\DESARROLLO SOFTWARE\\APP_WEB\\BackEnd\\routes\\producto.routes.js',
  'C:\\Users\\marti\\OneDrive\\Escritorio\\SOFTWARE\\DESARROLLO SOFTWARE\\APP_WEB\\BackEnd\\index.js'
]
}

Node.js v24.11.0
[nodemon] app crashed - waiting for file changes before starting...
[nodemon] restarting due to changes...
[nodemon] starting `node BackEnd/index.js`
server activo en el puerto 3000
DB se encuentra conectada
█
```

1. USUARIOS

1.1 crear usuario

Método: POST

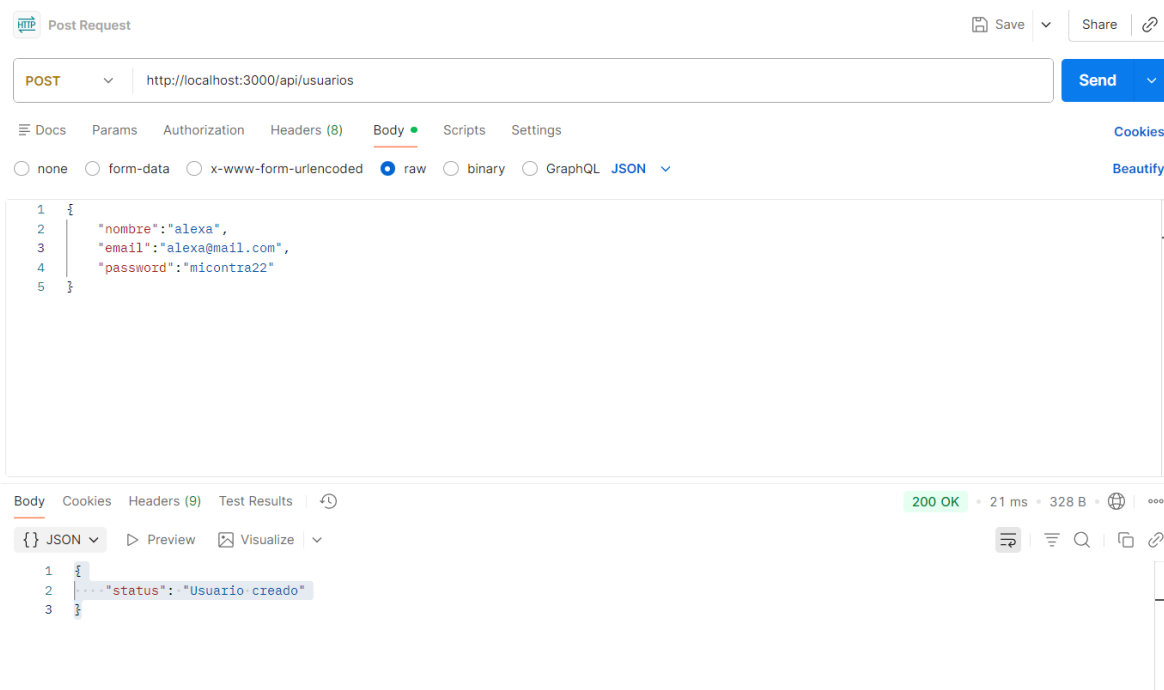
URL: <http://localhost:3000/api/usuarios>

Descripción: Crear un nuevo usuario en la base de datos.

Body (JSON):

```
{
  "nombre": "Alex",
  "email": "alex@example.com",
  "password": "miContra"
}
```

Resultado: Usuario creado exitosamente.



1.2 buscar usuario

metodo: GET

URL: <http://localhost:3000/api/usuarios>

Respuesta: devuelve un array con los usuarios registrados y sus_id

The screenshot shows a REST client interface with a GET request to `http://localhost:3000/api/usuarios`. The response is a 200 OK status with a response time of 51 ms and a body size of 412 B. The response body is displayed in JSON format, showing an array of user objects. The first user object is expanded, showing the following details:

```
1 [
2   {
3     "_id": "692b73ea60b341cdf97b17d",
4     "nombre": "alexa",
5     "email": "alexa@mail.com",
6     "password": "micontra22",
7     "__v": 0
8   }
9 ]
```

1.3 consulta de Id

método: GET

URL: <http://localhost:3000/api/usuarios/692b73ea60b341cdf97b17d>

Resultado: se obtiene la información del usuario correspondiente al ID suministrado.

The screenshot shows a REST client interface with a GET request to `http://localhost:3000/api/usuarios/692b73ea60b341cdf97b17d`. The response is a 200 OK status with a response time of 18 ms and a body size of 410 B. The response body is displayed in JSON format, showing a single user object with the following details:

```
1 {
2   "_id": "692b73ea60b341cdf97b17d",
3   "nombre": "alexa",
4   "email": "alexa@mail.com",
5   "password": "micontra22",
6   "__v": 0
7 }
```

. 1.4 Actualizar usuario

Metodo: POST

URL: <http://localhost:3000/api/usuarios/692b73ea60b341cdff97b17d>

Resultado: se modifican los campos de nombre y password del usuario existente, dando como resultado usuario actualizado

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** <http://localhost:3000/api/usuarios/692b73ea60b341cdff97b17d>
- Body (raw):**

```
1 {  
2   "nombre": "alexa ACTUALIZADAG",  
3   "email": "alexa@gmail.com",  
4   "password": "micontraNueva22"  
5 }
```
- Response (JSON):**

```
1 {  
2   "status": "Usuario actualizado"  
3 }
```
- Status:** 200 OK
- Time:** 24 ms
- Size:** 333 B

1.5 eliminar usuario

Metodo: DELETE

URL <http://localhost:3000/api/usuarios/692b73ea60b341cdff97b17d>

Resultado: el servuiidor confirma la eliminación con el mensaje “Usuario Eliminado”

The screenshot shows a REST client interface with the following details:

- Method:** DELETE
- URL:** <http://localhost:3000/api/usuarios/692b73ea60b341cdff97b17d>
- Body (raw):**

```
1 {  
2   "nombre": "alexa ACTUALIZADAG",  
3   "email": "alexa@gmail.com",  
4   "password": "micontraNueva22"  
5 }
```
- Response (JSON):**

```
1 {  
2   "status": "Usuario eliminado"  
3 }
```
- Status:** 200 OK
- Time:** 28 ms
- Size:** 331 B

2. PRODUCTOS

2.1 crear producto

método: POST

URL: <http://localhost:3000/api/productos>

Resultado: se envían los datos del producto en formato JSON y el servidor confirma la creación.

```
{
  "nombre": "camisa missie",
  "descripcion": "camisa elegante en satin",
  "precio": "120000",
  "stock": "20"
}
```

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** <http://localhost:3000/api/productos>
- Body:** A JSON object:

```
{
  "nombre": "camisa missie",
  "descripcion": "camisa elegante en satin",
  "precio": "120000",
  "stock": "20"
}
```
- Response:** 200 OK, 13 ms, 329 B. The response body is a JSON object:

```
{
  "status": "Producto creado"
}
```

2.2 buscar producto

método: GET

URL: <http://localhost:3000/api/productos>

Resultado: se consulta todos los productos creados

2.3

GET <http://localhost:3000/api/productos> Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

Document this request... Write with AI

Body Cookies Headers (9) Test Results 200 OK • 6 ms • 428 B

```
{
  "_id": "692c97bc60b341cdf97b184",
  "nombre": "camisa missie",
  "precio": 120000,
  "descripcion": "camisa elegante en satin",
  "__v": 0
}
```

2.3 consultar un producto Id

método: GET

URL: <http://localhost:3000/api/productos/692c97bc60b341cdf97b184>

Respuesta: se envia la solicitud con el ID del producto y el servicio retorna los datos del producto correspondiente.

GET <http://localhost:3000/api/productos/692c97bc60b341cdf97b184> Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

Document this request... Write with AI

Body Cookies Headers (9) Test Results 200 OK • 7 ms • 426 B

```
{
  "_id": "692c97bc60b341cdf97b184",
  "nombre": "camisa missie",
  "precio": 120000,
  "descripcion": "camisa elegante en satin",
  "__v": 0
}
```


2.4 actualizar producto

método: PUT

URL: <http://localhost:3000/api/productos/692c97bc60b341cdff97b184>

Respuesta: Actualizacion de un producto, se envían los nuevos datps del producto y el servidor responde con el mensaje. “producto Actualizado”

The screenshot shows a REST client interface with a PUT request to `http://localhost:3000/api/productos/692c97bc60b341cdff97b184`. The request body is a JSON object with the following fields: `nombre` (vestido missie), `descripcion` (version actualizada con mejoras en costura), `precio` (230000), and `stock` (20). The response is a 200 OK status with a response time of 13 ms and a body size of 334 B. The response body is a JSON object with the field `status` (Producto actualizado).

```
PUT http://localhost:3000/api/productos/692c97bc60b341cdff97b184
```

```
{
  "nombre": "vestido missie",
  "descripcion": "version actualizada con mejoras en costura",
  "precio": "230000",
  "stock": "20"
}
```

200 OK • 13 ms • 334 B • 🌐 • ⋮

```
{
  "status": "Producto actualizado"
}
```

2.5 eliminar producto

método: DELETE

URL: <http://localhost:3000/api/productos/692c97bc60b341cdff97b184>

Resultado: el servidor responde con el mensaje”Producto Eliminado”

The screenshot shows a REST client interface with a DELETE request to `http://localhost:3000/api/productos/692c97bc60b341cdff97b184`. The response is a 200 OK status with a response time of 17 ms and a body size of 332 B. The response body is a JSON object with the field `status` (Producto eliminado).

```
DELETE http://localhost:3000/api/productos/692c97bc60b341cdff97b184
```

Document this request... ✎ Write with AI

200 OK • 17 ms • 332 B • 🌐 • ⋮

```
{
  "status": "Producto eliminado"
}
```

3. LOGIN:

Método: POST

URL: <http://localhost:3000/api/usuarios/login>

Resultado: se autentico el login.

The screenshot displays a REST client interface with the following details:

- Request:**
 - Method:** POST
 - URL:** `http://localhost:3000/api/usuarios/login`
 - Body:** A JSON object with the following structure:

```
1 {
2   "email": "alexagh@gmail.com",
3   "password": "micontra"
4 }
```
- Response:**
 - Status:** 200 OK
 - Time:** 10 ms
 - Size:** 451 B
 - Body:** A JSON object with the following structure:

```
1 {
2   "message": "Autenticado",
3   "usuario": {
4     "_id": "692ca0eb60b341cdf97b194",
5     "nombre": "alexandra",
6     "email": "alexagh@gmail.com",
7     "password": "micontra",
8     "__v": 0
9   }
10 }
```

4. Autenticador

En este apartado se implementa la funcionalidad de autenticación del usuario.

Cuando un usuario ingresa su correo y contraseña, el sistema valida las credenciales, y si son correctas

The screenshot shows a REST client interface with a POST request to `http://localhost:3000/api/auth/regar`. The request body is a JSON object with the following fields:

```
1 {
2   "nombre": "dannaa",
3   "email": "dannaaagh@gmail.com",
4   "password": "micontra"
5 }
```

The response is a 201 Created status with a 168 ms response time and 407 B of data. The response body is a JSON object:

```
1 {
2   "msg": "Usuario registrado correctamente",
3   "usuario": {
4     "id": "693c56bfa7718a494961af0e",
5     "nombre": "dannaa",
6     "email": "dannaaagh@gmail.com"
7   }
8 }
```

The screenshot shows the MongoDB Compass interface. The left sidebar shows the database structure with the following connections:

- MUNDIAL
- localhost:27017
 - MUNDIAL
 - admin
 - config
 - empleados_085
 - productos
 - usuarios
 - local

The main panel shows the 'usuarios' collection in the 'empleados_085' database. The documents are as follows:

```
1 {
2   "_id": ObjectId('692ca08760b341cdff97b18c'),
3   "nombre": "alexa",
4   "email": "alexa@gmail.com",
5   "password": "micontra",
6   "__v": 0
7 }
8
9 {
10  "_id": ObjectId('692ca0eb60b341cdff97b194'),
11  "nombre": "alexandra",
12  "email": "alexag@gmail.com",
13  "password": "micontra",
14  "__v": 0
15 }
16
17 {
18  "_id": ObjectId('693c56bfa7718a494961af0e'),
19  "nombre": "dannaa",
20  "email": "dannaaagh@gmail.com",
21  "password": "$2b$10$4.iiitmh4cHuAh.c56waqmuC/IwV5qo18X2G/05EzCw2P7Pj5d3iKy",
22  "__v": 0
23 }
```

POST

http://localhost:3000/api/auth/login

Send

Docs

Params

Authorization

Headers (8)

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1

2

3

4

{

"email": "dannaagh@gmail.com",

"password": "micontra"

}

Body

Cookies

Headers (8)

Test Results

200 OK

134 ms

383 B

{}

JSON

Preview

Visualize

1

2

3

4

5

6

7

8

{

"msg": "Login exitoso",

"usuario": {

"id": "693c56bfa7718a494961af0e",

"nombre": "danna",

"email": "dannaagh@gmail.com"

}

}

POST

http://localhost:3000/api/auth/login

Send

Docs

Params

Authorization

Headers (8)

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1

2

3

4

{

"email": "dannaagh@gmail.com",

"password": "micontra"

}

Body

Cookies

Headers (8)

Test Results

400 Bad Request

8 ms

307 B

{}

JSON

Preview

Debug with AI

1

2

3

{

"msg": "Usuario no encontrado"

}

6. URL del Proyecto

Se creó un repositorio público en GitHub llamado APPI-CRUD, en el cual se cargó todo el proyecto desarrollado.

El repositorio contiene la estructura completa del CRUD, incluyendo archivos, conexión a base de datos, estilos, scripts y recursos utilizados en la práctica.

Finalmente, se verificó la correcta publicación del proyecto desde GitHub Desktop, confirmando que los archivos quedaran disponibles en la nube para su revisión y seguimiento.

URL: [Castilloyurani91-blip/APPI_CRUD](https://github.com/Castilloyurani91-blip/APPI_CRUD)

CONCLUSIONES

La API desarrollada cumple con los requisitos solicitados. Se implementó un CRUD completo para usuarios y productos utilizando Node.js, Express y MongoDB. Se realizaron pruebas con Postman y se aplicó una estructura MVC adecuada que facilita el mantenimiento del proyecto.